

Design and Analysis of Algorithms

Semester Project Proposal

UstaadAssist: Adaptive Semester Planning and Replanning for University Teachers

Domain / Application Area: Academic course operations — semester timetabling, disruption replanning, assessment scheduling and grading

Group 07

Seat Number	Name
EB24210106051	Muhammad Sajeel Hassan (Group Leader)
EB24210106077	Muhammad Meesum Ali
EB24210106119	Syed Muhammad Yaamin Ali
EB24210106014	Affan Bin Rashid
EB24210106089	Neha Nasir
EB24210106094	Ramsha Mirza

Course: Design and Analysis of Algorithms · Department of Computer Science, University of Karachi · **Submission date:** 28 September 2026

Central Question of the Project

There is no single best algorithm for every decision a semester forces on a university teacher. Laying topics onto class dates, recovering when classes are lost, placing quizzes after the material they test, and rebuilding the rest of the term after a disruption are different optimization problems with different structures. Our task is to design, implement, analyze and experimentally compare different algorithmic approaches to each of them, determine which approach is most suitable under different real conditions (a normal term, a term that has lost several classes, a very large synthetic course), and integrate the working algorithms into a functional application that demonstrates them end to end.

1. Problem Statement

A university teacher runs every course as a small scheduling system. A semester has a **fixed start and end date** and a **fixed set of weekly class days**. Removing public holidays leaves a limited number of real class dates, typically 25 to 40. Into those dates the teacher must fit a syllabus of 10 to 40 topics, each needing one or more classes, some more important than others. Around that timetable sit quizzes, assignments, a midterm and a final, each of which may only be held after the topics it covers have been taught, plus the attendance and marks of every enrolled student.

The system therefore involves many entities competing for one scarce resource, the class slot, under hard time constraints: the semester end date cannot move, and an assessment must never precede its topics. That makes it a natural fit for algorithmic optimization.

This project designs **UstaadAssist**, an adaptive semester planning system. Its core entities are the **course**, its **topics** (classes needed, minimum classes, priority), the **class slots** (real teaching dates), **public holidays**, **assessments** and the topics they cover, **makeup days**, and the **students** with their attendance

and marks.

The project requires building a working application

The main objective is to apply Design and Analysis of Algorithms (DAA) concepts to a real-world optimization problem: implement multiple algorithmic solutions, analyze their computational complexity, experimentally compare their performance, and integrate these algorithms into a working application that a user can actually run and interact with. The algorithms are the graded core of the project, but they are demonstrated inside a real, usable system rather than as disconnected scripts: a mobile application for the teacher, backed by a REST service that runs every algorithm described here.

2. Scenario Description

A teacher runs one course for one semester, with the following entities and constraints:

- **A fixed, limited number of class slots:** the weekly class days between the start and end dates, minus public holidays. Every lost class permanently removes one.
- **Many competing topics:** 10 to 40 per course, all of which want slots, and which must be taught in a sensible order.
- **Topics of different sizes:** each needs a number of classes (its load), and has a minimum below which it cannot be meaningfully taught.
- **Different priority levels:** each topic is high, normal or low priority.
- **Deadlines:** the semester end date is fixed; each assessment has a scheduled date.
- **Dependent entities:** every assessment depends on the topics it covers and must fall after the last of them is taught.
- **Secondary resources:** makeup days (days the course is not normally taught, excluding Sundays and holidays) that can be added only at a cost to the teacher.
- **Limited overall capacity:** the total teaching the syllabus needs can exceed the slots available, producing a **deficit**.

Unexpected events are routine: classes are cancelled (campus closures, strikes, exam duty, illness), public holidays fall on class days, and topics overrun their planned number of classes. Each event can make the original plan **inefficient or infeasible**: topics no longer fit, and quizzes fall before their material. The system must help the teacher respond using algorithms, rebuilding the future of the plan without disturbing what has already been taught.

3. Main Objective

The system must answer the following algorithmic decision questions:

- **Assignment:** on which class date should each topic be taught?
- **Prioritization under scarcity:** when the remaining topics need more classes than remain, which topics should be dropped, which shortened, or which makeup dates added?
- **Constraint satisfaction:** on which date can each assessment be held so that it never precedes its topics, with the least delay?
- **Replanning:** after a disruption, how should the rest of the semester be rebuilt while never changing a class that has already been taught, and what exactly moved?
- **Monitoring:** how far behind its plan is the course?
- **Reliable input:** how can the class list be read from a phone photograph with the fewest misread rows?

Different algorithmic techniques are used to answer these questions rather than a single fixed method: greedy allocation, dynamic programming, binary search, hashing, incremental recomputation and majority voting. These techniques are integrated into a single working application in which the user can exercise each decision point.

4. Required Problem Components

The project selects five optimization components, meeting the minimum of four. Components A, B, C and F follow the template directly. Component E is adapted to our domain as a multi-stage pipeline.

Component D (network route optimization) is not selected. Semester planning has no physical network to route through. The one natural graph in the domain, prerequisites between topics, would require the teacher to enter every dependency by hand, which contradicts the system's design rule of asking for the minimum input. The five components below cover the domain's real optimization problems.

In every component, the algorithms are **pure functions**: data in, data out, with no database access. The application loads the data, calls the algorithm, and saves the result, which makes each algorithm independently testable and measurable.

Component A — Class Slot Assignment (Resource Assignment)

A limited number of class slots must be assigned to competing topics. First the slots are generated: every date from start to end whose weekday is a class day and which is not a holiday. Then topics are assigned to slots. Each topic has:

- Topic ID and teaching order
- Classes needed (its load) and minimum classes
- Priority (High / Normal / Low)
- Assigned start and end date (the output)

Sample data: the planner's real output for a Database Systems course, Mondays and Wednesdays, 1 September to 18 December 2026. Defence Day (6 Sep) and Iqbal Day (9 Nov) are removed, leaving 30 slots for 30 needed classes.

Topic	Order	Classes needed	Minimum	Priority	Planned start	Planned end
Introduction to Database Systems	1	1	1	Normal	2 Sep	2 Sep
The Relational Model	2	2	1	High	7 Sep	9 Sep
Entity Relationship Modelling	3	3	2	High	14 Sep	21 Sep
Relational Algebra	4	2	1	Normal	23 Sep	28 Sep
SQL: Queries and Joins	5	4	3	High	30 Sep	12 Oct
Normalization	8	3	2	High	28 Oct	4 Nov
Transactions and ACID	11	2	2	High	30 Nov	2 Dec
Recovery and Backup	13	1	1	Low	14 Dec	14 Dec

Approach A1: Sequential greedy (teaching order). Walk the topics in teaching order with one pointer into the slot list; each topic takes the next **classes needed** slots. A topic is never split: one that does not fit whole goes to an overflow list.

```
ALLOCATE-SEQUENTIAL(slots, topics):
    sort topics by order_no
    next = 0
    for each topic t:
        if next + t.need > |slots|: overflow.add(t); continue
        for part = 1 .. t.need:
            sessions.add(slots[next], t, part); next = next + 1
    return sessions, overflow
```

Approach A2: Priority-first allocation. When the slots cannot hold every topic, decide **which** topics fit by priority (high first, then normal, then low), and only then lay the chosen topics out in teaching order. This guarantees that the most important topics are scheduled, at the cost of possibly skipping a foundation topic of lower priority.

Comparison. Both run in $O(n \log n + S)$. A1 preserves the pedagogical order exactly and is optimal whenever everything fits, which is the normal case. A2 only differs under a deficit, where A1 overflows the **last** topics regardless of importance while A2 overflows the **least important**. Experiments measure the priority-weighted syllabus coverage of each under increasing deficits.

Component B — Deficit Resolution (Scheduling Under Constraints)

More teaching is needed than the semester can hold:

```
deficit = classes the remaining topics need - class slots still available
```

Each remaining topic has a load (classes needed), a minimum, and a priority; the cost of the deficit is the teaching that must be given up. **Objective:** recover at least **deficit** classes while losing as little as possible. Rather than failing, the system computes three options and the teacher chooses one: **Drop** whole topics, **Compress** topics towards their minimum, or **Extend** with makeup days.

Drop, Approach B1: Greedy by priority. Sort topics least important first (later topics first among equals) and take topics until the deficit is covered. $O(n \log n)$.

Drop, Approach B2: 0/1 knapsack by dynamic programming. Each topic is either dropped entirely or kept (0/1), and frees a known number of classes (its weight). The cost of a set of dropped topics is compared lexicographically: first the number of high-priority topics dropped, then normal, then low (the fewest topics), then the fewest classes beyond the deficit, then a preference for later topics.

```
KNAPSACK-DROP(topics, deficit):
    T = total classes of all topics
    best[0][0] = zero cost; best[0][s] = impossible for s > 0
    for i = 1 .. n:
        for s = 0 .. T:
            skip = best[i-1][s]
            drop = best[i-1][s - w(i)] + cost(topic i)    if s >= w(i)
            best[i][s] = cheaper of skip and drop; took[i][s] = (drop chosen)
    s* = the cheapest best[n][s] with s >= deficit (smaller s breaks ties)
    walk back from (n, s*) through took[][] to list the dropped topics
```

Why dynamic programming is justified here. Greedy is not optimal for the 0/1 problem. Short by 3 classes, with low-priority topics of 3, 2 and 2 classes, greedy takes 2 + 2 (two topics, one class too many)

while dropping the 3-class topic alone is exact. DP examines every useful combination through the table without enumerating 2^n subsets.

Compress, Approach B3: Greedy by largest slack, where slack = classes needed minus minimum.

Taking classes from the topic with the most slack first is **provably optimal**, because every class removed has equal value and a topic may give up any amount up to its slack (the fractional structure). **Approach B4: exhaustive search** over reductions serves as the optimality baseline in experiments. **Extend** takes the earliest **deficit** dates that are not class days, Sundays or holidays.

Component C — Assessment Placement (Contention / Slot Allocation)

Several assessments contend for class dates, each constrained to fall after the last class of every topic it covers. Factors: the assessment's scheduled date, the topics it covers, how far those topics have slipped, and the assessment type. **Objective:** zero conflicts (no assessment before its topics) and minimum delay (each conflicting assessment moves to the **first** valid class date, not an arbitrary later one). Every move carries a reason for the teacher, for example "Quiz 3 moved from 18 Sep to 25 Sep because Normalization will not be completed before 24 Sep."

One pass over the plan builds a hash map from each topic to its last teaching date. For each assessment, the latest of these among its topics is the constraint date **c**, and the question becomes: which is the first slot strictly after **c**?

Approach C1: Linear scan from the start of the slot list, $O(S)$ per assessment.

Approach C2: Binary search on the date-ordered slot list, $O(\log S)$ per assessment:

```
FIRST-SLOT-AFTER(slots, c):
    low = 0; high = |slots|
    while low < high:
        mid = floor((low + high) / 2)
        if slots[mid].date <= c: low = mid + 1
        else: high = mid
    return slots[low] if low < |slots| else NONE
```

The slot list is sorted by construction, so binary search applies with no preprocessing. Both approaches return identical answers (verified exhaustively over every date of a semester); they differ only in cost as S grows.

Component F — Replanning Under Disruption (Overall Resource Allocation)

The system's resources are the **remaining class slots** (the primary bottleneck), **makeup days** (a secondary resource, costly to the teacher), and the **slack between each topic's needed and minimum classes** (a reserve that Compress can draw on). A cancellation or a topic overrun changes the balance between them and triggers a replan. The replan must obey one invariant, **freeze the past**: a conducted class is never altered.

Approach F1: Full regeneration. Discard the plan and run slot generation and allocation from the semester start, as at setup.

Approach F2: Incremental freeze-and-rebuild. Keep every conducted session; compute each topic's remaining need (classes needed minus classes already given); generate slots only from today to the end date, keeping the course's week numbering; allocate the remaining work; re-run assessment placement; then compare the old and new plans topic by topic to report exactly what moved.

```

REPLAN(existing, topics, today):
    frozen = sessions with status = conducted
    remaining(t) = t.need - (frozen sessions of t)    for each topic t
    slots = GENERATE-SLOTS(today, end, class_days, holidays)
    sessions = frozen + ALLOCATE(slots, topics with remaining > 0)
    moves = PLACE-ASSESSMENTS(sessions, slots, assessments)
    changes = DIFF(old plan, new plan)    (per topic, dates paired in order)

```

Bottlenecks and scaling. Class slots are the binding constraint: each lost class raises the deficit by one, and once the deficit is positive the resolution cost (Component B) grows with $n \times T$. Schedule health is monitored alongside:

```

behind_by_weeks = (classes planned up to today - classes conducted) / classes per week

```

Comparison. F1 is simpler, but it is not merely slower: it violates the invariant, because regenerating from the semester start reassigns topics to dates that have already been taught. F2 is correct and bounds its work to the future part of the semester. Experiments measure run time and **disruption**, the number of future sessions whose date changes.

Component E (adapted) — Multi-Stage Pipeline: Class List Extraction

Students are never typed. The class list passes through stages: photograph, OCR text recognition, parsing of seat numbers (EB followed by 11 digits) and names, reconciliation, then a mandatory teacher review before anything is saved. The objective is the fewest misread rows at an acceptable processing time, since one misread seat number would silently attach a student's attendance and marks to the wrong person for the whole term.

Approach E1: Single OCR reading, parsed directly. Fastest.

Approach E2: Majority vote over six readings (three page-layout modes of the Tesseract OCR engine, each with and without automatic rotation). Rows are matched across readings by seat number and occurrence; for each row the name spelling most readings agree on wins, with spellings differing only by spaces counted as one vote; rows with weak agreement are flagged with low confidence for the review screen.

```

VOTE(readings):
    for each reading r, for each parsed row (seat, name) in r:
        key = seat + "#" + occurrence of seat in r
        tally[key].seen += 1; tally[key].votes[name without spaces] += 1
    for each key: name = most-voted spelling;
        confidence = high only if seen and agreement >= 2 and no rival spelling

```

On the review screen, duplicate seat numbers are found with a hash map in $O(m)$ for m students, rather than by pairwise comparison in $O(m^2)$.

Comparison. E2 costs roughly six times the OCR time of E1 in exchange for fewer undetected misreads. Experiments measure rows correct, rows flagged and time per page.

5. Algorithm Comparison — Core Requirement

At least two approaches are implemented for each major comparable problem and compared against each other on the same inputs.

Component	Approach 1	Approach 2
A. Class slot assignment	Sequential greedy (teaching order)	Priority-first allocation
B. Deficit resolution: Drop	Greedy by priority	0/1 knapsack (dynamic programming)
B. Deficit resolution: Compress	Greedy by largest slack	Exhaustive search (optimality baseline)
C. Assessment placement	Linear scan	Binary search
F. Replanning after disruption	Full regeneration	Incremental freeze-and-rebuild
E. Class list extraction	Single OCR reading	Majority vote over six readings

For each pair, the project determines which algorithm performs better, under what conditions, and why. The hypotheses the experiments will test:

- **Drop:** greedy and DP give identical results when topic sizes are uniform; DP is strictly better (fewer topics lost, fewer classes over-recovered) when sizes are uneven. Greedy is faster, $O(n \log n)$ against $O(n \times T)$, but both are well under a millisecond at semester scale, so DP is the right choice.
- **Compress:** greedy by slack matches the exhaustive optimum on every input, while the exhaustive search grows exponentially. This demonstrates when greedy is provably optimal.
- **Assessment placement:** both find the same dates; binary search wins as the slot list grows, and the difference is negligible at 30 to 40 slots.
- **Replanning:** incremental rebuild is the only approach that preserves taught classes, and it changes fewer future sessions than full regeneration.
- **Assignment:** sequential allocation is best when everything fits; priority-first covers more priority-weighted syllabus under a deficit, at the cost of teaching order.
- **Extraction:** voting reduces undetected misreads at roughly six times the processing time; a text-based department PDF is exact with either approach.

6. Input Size Experimentation

All algorithms are tested with datasets of increasing size. A real semester is the Small case; the larger datasets are generated synthetically with a fixed random seed, so every run is reproducible, to expose asymptotic behaviour.

Dataset	Topics	Class slots	Assessments	Students (class list)
Small	20	40	5	20
Medium	100	200	15	60
Large	500	1,000	30	120
Very Large	1,000	2,000	50	200

For deficit experiments, a deficit of 10% of the total classes needed is imposed by removing slots. For replanning experiments, a random future class is cancelled after a random number of classes have been conducted.

The following metrics are recorded, where applicable:

- **Execution time** (median of repeated runs)
- **Number of operations**: comparisons, DP table cells filled, binary-search probes
- **Total cost**: topics dropped and classes recovered beyond the deficit (Drop and Compress)
- **Total delay**: days each moved assessment is pushed back
- **Conflicts**: assessments left before their topics (must be zero)
- **Disruption**: future sessions whose date changes after a replan
- **Accuracy**: class list rows read correctly, and rows flagged for review
- **Memory usage**: DP table size, and heap usage measured by the runtime

7. Theoretical Complexity Analysis

Notation: **D** days in the semester, **S** class slots, **n** topics, **T** total classes the topics need (the sum of their loads), **A** assessments, **k** topics covered per assessment, **H** holidays, **R** OCR readings (six), **L** lines of recognised text, **m** students.

Algorithm	Best	Average	Worst	Space
Slot generation (hash-set holidays)	$O(D)$	$O(D)$	$O(D)$	$O(S + H)$
A1 Sequential allocation	$O(n \log n + S)$	$O(n \log n + S)$	$O(n \log n + S)$	$O(n + S)$
A2 Priority-first allocation	$O(n \log n + S)$	$O(n \log n + S)$	$O(n \log n + S)$	$O(n + S)$
B1 Greedy drop	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
B2 Knapsack drop (DP)	$O(n \times T)$	$O(n \times T)$	$O(n \times T)$	$O(n \times T)$
B3 Greedy compress	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
B4 Exhaustive compress	$O(2^n \times n)$	$O(2^n \times n)$	$O(2^n \times n)$	$O(n)$
Extend (makeup dates)	$O(\text{deficit})$	$O(D)$	$O(D)$	$O(\text{deficit})$
C1 Placement, linear scan	$O(S + A \times k)$	$O(S + A \times (k + S))$	$O(S + A \times (k + S))$	$O(n)$
C2 Placement, binary search	$O(S + A \times (k + \log S))$	same	same	$O(n)$
F1 Full regeneration	$O(D + n \log n + S + A \log S)$	same	same	$O(n + S)$
F2 Incremental replan	$O(D + n \log n + S \log S + A \log S)$	same	same	$O(n + S)$
E1 Single reading	$O(L)$	$O(L)$	$O(L)$	$O(L)$
E2 Majority vote	$O(R \times L)$	$O(R \times L)$	$O(R \times L)$	$O(R \times L)$
Duplicate seat check (hash map)	$O(m)$	$O(m)$	$O(m)$	$O(m)$
Schedule health	$O(S)$	$O(S)$	$O(S)$	$O(1)$

Why these forms.

- **Slot generation** visits every calendar day exactly once, and each holiday test is a constant-time hash-set lookup instead of a scan of the holiday list, so the cost is linear in D regardless of H .

- **Allocation** is dominated by sorting the topics; after that a single pointer moves forward through the slots and never moves back, so each slot is visited at most once.
- **Knapsack** fills a table of $(n + 1) \times (T + 1)$ cells, each in constant time from two cells of the previous row; reading back the chosen topics adds $O(n)$. There is no early exit, so best, average and worst cases coincide. The bound is **pseudo-polynomial**: polynomial in the value T rather than in the input's length. It is small in practice because T is a count of classes (about 30 to 40 per semester), and it is exponentially cheaper than the $O(2^n)$ of trying every subset.
- **Linear scan placement** can stop at the first slot (best case) but must, on average, pass about half the slots; **binary search** halves the remaining range on every probe, so it takes about $\log_2(S)$ probes in every case.
- **Incremental replan** adds an $O(S \log S)$ term over allocation because the report of what moved sorts each topic's old and new dates before pairing them.
- **Voting** parses each of the R readings once and tallies rows in a hash map, so it is linear in the total text. OCR recognition itself dominates the wall-clock time and is measured experimentally rather than analysed.

8. Application Requirement

The project is a working application in which every algorithm runs on real data:

- **Mobile application** (React Native with Expo) for the teacher: course setup, the teaching plan, attendance, marks, grading and reports.
- **REST service** (Node.js, Express and PostgreSQL, deployed on Vercel) that runs every algorithm on each request. Sign-in and file storage use Supabase.
- **Automated tests** (Node's built-in test runner) for every planner algorithm, including a check of the knapsack against exhaustive search on 500 random cases.

How the user inputs data and triggers each component in the application:

Component	User action in the app
A. Slot assignment	Enter dates, class days and topic titles; tap Generate my plan
B. Deficit resolution	Catch-up options on the Plan tab: three computed options (Drop, Compress, Add makeup classes); tap Choose
C. Assessment placement	Create an assessment and choose its topics; any conflicting date is moved with a written reason
F. Replanning	Mark a class cancelled with a reason, or change a topic's classes needed; the "what moved" screen shows every change
E. Class list extraction	Photograph the class list or upload the department PDF; check the rows on the review screen
Schedule health	The warning on the course dashboard, e.g. "You are 1 week behind your plan"

For algorithm comparison and performance analysis, the project provides a menu-driven command-line tool that calls the **same** planner functions the application uses, with no placeholders or hard-coded output:

```

=====
    USTAADASSIST PLANNER OPTIMIZER
=====
1. Class Slot Assignment
2. Deficit Resolution (Drop / Compress / Extend)
3. Assessment Placement
4. Replanning After Disruption
5. Class List Extraction
6. Schedule Health
7. Algorithm Comparison
8. Performance Analysis

```

9. Minimum Requirements Checklist

Requirement	Where it is met
1. At least four optimization components	Five: A, B, C, E (adapted) and F, in Section 4
2. Two approaches per comparable problem	Six comparison pairs, Section 5
3. Algorithmic idea of each approach	Section 4, per component
4. Pseudocode	Section 4: allocation, knapsack, binary search, replanning, voting
5. Implementation in code	Pure planner functions in the backend, called by the application
6. Time complexity	Section 7, best, average and worst case
7. Space complexity	Section 7
8. Theoretical against experimental performance	Section 6 datasets and metrics, compared with Section 7
9. Advantages and limitations	Section 4 comparisons and Section 5 hypotheses
10. Most appropriate algorithm per scenario	Section 5
11. Working application integrating all algorithms	Section 8: mobile app, REST service and command-line optimizer